# Racetrack Assignment: Neural Network (Optional Extension)

## Overview

**Note:** This portion of the Racetrack project is optional and ungraded. You may complete it if you wish to explore neural network control using Unity ML-Agents.

The goal is to train a neural network to drive the racecar autonomously using reinforcement learning. You will use Unity's `ml-agents` package, which provides the necessary tools to train and evaluate agents in simulated environments.

## Setup

- Your agent is `NNVehicle.cs` under `Scripts/GameAIStudentWork/`. It observes the crisp inputs your `FuzzyVehicle` reports from `CollectCrispInputs()`, which you write, and you can add observations of your own. It has no reward yet, and designing one is your main task for reinforcement learning (the `TODO (reward)` items).
- The agent is on the prefab `Assets/Prefab/RacecarNNAgent.prefab`, which the four NN scenes (such as `RaceTrackNN`) and the tests use.

## Step by Step

Your network learns from your `FuzzyVehicle` first (imitation), then improves on its own, driven by the reward you design (reinforcement learning). Your design decisions go into steps 2 and 7. The sections that follow explain each step.

1. Set up the trainer, once (*One-time setup* below).
2. **What the network observes.** In your `FuzzyVehicle`, fill in `CollectCrispInputs()` with the crisp values your fuzzy rules evaluate (for example, speed, etc.; see *Observations*). On the `RacecarNNAgent` prefab, set *Behavior Parameters* > *Vector Observation* > *Space Size* to the number of values.
3. `Tools/ml/nntools build`
4. **How large each input is.** Record briefly: `Tools/ml/nntools record --seconds 120 --name Sizing`. The end of each track's report gives a typical largest value of each crisp input. Enter those, in order, as *NN Vehicle* > *Observation Scales* on the prefab, rebuild (step 3), and delete `Demonstrations/Sizing`.
5. `Tools/ml/nntools record`, and read the report for each track: is your controller a good teacher, and does the network see values roughly in $[-1, 1]$?
6. Imitation, 400k steps. It learns from your `FuzzyVehicle` alone, so it needs no reward yet:

```
Tools/ml/nntools train --config Tools/ml/config/Racecar_imitation.yaml \
    --mode headless --num-envs 4 --scene auto --run-id imitation
```

Windows (PowerShell):

```
Tools\ml\nntools.cmd train --config Tools/ml/config/Racecar_imitation.yaml `
    --mode headless --num-envs 4 --scene auto --run-id imitation
```

Test it: `Tools/ml/nntools test --nn --run imitation --steps 200000 300000 400000`. It should drive roughly like your `FuzzyVehicle`.

7. **The reward.** Design it in `NNVehicle` (see *Reward Structure*), put its settings in `Racecar_rl_after_imitation.yaml`, and rebuild.
8. Reinforcement learning from the imitation policy, 1.5M steps:

```
Tools/ml/nntools train --config Tools/ml/config/Racecar_rl_after_imitation.yaml \
    --mode headless --num-envs 4 --scene auto --run-id rl \
    -- --initialize-from imitation
```

Windows (PowerShell):

```
Tools\ml\nntools.cmd train `
    --config Tools/ml/config/Racecar_rl_after_imitation.yaml `
    --mode headless --num-envs 4 --scene auto --run-id rl `
    -- --initialize-from imitation
```

9. `Tools/ml/nntools test --nn --run rl` compares checkpoints in the graded races. Keep the best, and compare it with your `FuzzyVehicle` (`Tools/ml/nntools test`). Then revise the reward (step 7) and train again.

Steps 1-6 take about half an hour. A reinforcement learning run takes about 45 minutes on a fast desktop, longer on a laptop; for a quick reward experiment, stop it early (Ctrl+C) and test the checkpoints it saved so far.

## Workflow and Tools (Draft)

The project includes tools for the repetitive parts: setting up the trainer, building a training version of the game, recording demonstrations of your fuzzy logic driver, training, and testing. They work on macOS, Windows, and Linux. Run them from the project folder; `build` and `test` need the Unity Editor closed.

### One-time setup

The trainer (`mlagents-learn`) needs Python 3.10 (3.10.1–3.10.12, 64-bit); newer 3.10 releases are rejected. Install it (python.org, `pyenv`, conda, or `uv python install 3.10.12`). On Windows, python.org's last 3.10 installer is 3.10.11. Then create the project's Python environment:

```
macOS / Linux:  Tools/ml/setup_mlagents_env.sh
Windows:        Tools\ml\setup_mlagents_env.cmd
```

To use a particular Python, first set the environment variable `PYTHON` to its path.

All other commands go through `Tools/ml/nntools` (Windows: `Tools\ml\nntools.cmd`); add `--help` to any command for its options. The examples below use the macOS/Linux form. On Windows,

use `Tools\ml\nntools.cmd` and, in PowerShell, end a continued line with a backtick (`) instead of `\`; in Command Prompt, type the whole command on one line.

### Build the training game

```
Tools/ml/nntools build
```

This builds the four NN scenes (one per track) into `Builds/NNTraining/`. Rebuild whenever you change your code, the scenes, or the `RacecarNNAgent` prefab: training and recording run this build, not the Editor.

### Record demonstrations

```
Tools/ml/nntools record
```

Your `FuzzyVehicle` drives each track (all four at once, without graphics, as fast as your computer allows), and the recordings go to `Demonstrations/Racecar`. A report per track says how much was kept and whether your controller makes a good teacher; see the section *Learning from Your Fuzzy Logic Driver (Imitation)*. To record again, delete `Demonstrations/Racecar` first, add `--append` to add to it, or use `--name` for a separate set (and point `demo_path` in the imitation configuration at it). You can also record in the Editor: enable `record` on the `DemonstrationSession` component of the NN agent object and press Play.

### Train

```
Tools/ml/nntools train --config Tools/ml/config/Racecar.yaml \
    --mode headless --num-envs 4 --scene auto --run-id myrun
```

Windows (PowerShell):

```
Tools\ml\nntools.cmd train --config Tools/ml/config/Racecar.yaml `
    --mode headless --num-envs 4 --scene auto --run-id myrun
```

- `--mode headless` runs without graphics (fastest); `--mode windowed` shows small low-quality windows so you can watch; `--mode editor` trains in the Unity Editor (press Play when prompted).
- `--num-envs 4 --scene auto` runs four copies of the game at once, one per track. More copies collect experience faster if your computer has the cores for it.
- Results (checkpoints, the trained model) go to `results/<run-id>`. Watch progress with TensorBoard from the project's Python environment, no activation needed: `.venv-mlagents/bin/tensorboard --logdir results` (Windows: `.venv-mlagents\Scripts\tensorboard.exe --logdir results`).
- Arguments after `--` go to `mlagents-learn`, e.g. `-- --initialize-from otherrun` to start from another run's policy, or `-- --resume` to continue a stopped run.

### Test in the graded races

```
Tools/ml/nntools test --nn --run rl
```

Runs the graded races (the same tracks, durations, and targets as the fuzzy logic part) and prints each race's speed, wipeouts, and estimated score. `--run` tests the checkpoints of a training run nearest every 500k steps; pick others with `--steps 1000000 1500000`, or test any model file with `--model FILE.onnx`. Without `--nn` it runs your `FuzzyVehicle` for comparison. The same races are in the Test Runner (PlayMode, categories `FL` and `NN`); the `NN` tests use the model on the `RacecarNNAgent` prefab.

### Use a trained model in Unity

Unity only sees files under `Assets`, so first copy the model there: for example `results/<run-id>/Racecar/Racecar-<step>.onnx` (or the final `Racecar.onnx`) to `Assets/Models/Racecar.onnx`. Avoid `Assets/NNModels`, which the tests overwrite and Git ignores. Then open `Assets/Prefab/RacecarNNAgent.prefab`, drag the model from the Project window onto the `Model` field of `Behavior Parameters`, open an NN scene, and press Play. Set it on the prefab rather than on the agent object in one scene, so that every NN scene and `nntools test --nn` use it.

### Trainer configurations

- `Racecar.yaml`: reinforcement learning only, no demonstrations.
- `Racecar_imitation.yaml`, then `Racecar_rl_after_imitation.yaml`: learn from your demonstrations first, then improve with reinforcement learning (the section *Training in Two Phases*).

Settings under `environment_parameters` at the end of each file are read at the start of every episode, so you can change them without rebuilding the game: `wipeout_ends_episode` by `NNVehicle`, `rewind_fraction` by `WipeoutRewind`, and any reward settings you add and read in `NNVehicle.OnEpisodeBegin()`.

## Training Tips and Resources

It is strongly recommended that you read through the Unity ML-Agents introductory documentation before beginning. The following links may assist you in configuring and training your agent:

### ML-Agents Documentation

The project includes the `ML-Agents 4.1.0` package.

- Unity ML-Agents 4.1 Manual
- ML-Agents 4.1 Changelog

### Algorithm Background

- Proximal Policy Optimization (PPO) — overview of the reinforcement learning algorithm used by ML-Agents.

### Network Architecture

- Configuring Layers and Nodes in a Neural Network — guidelines for selecting network size and depth.

### Reward Design

- ML-Agents: Environment and Reward Design

### Performance and Optimization

- Using an Environment Executable — speeds up training by running Unity headless or in low-graphics mode.
- The `nntools train` modes above already run the game headless or in small low-quality windows.
- Training Using Concurrent Unity Instances — run multiple simulations in parallel to accelerate learning.

## Guidelines and Common Pitfalls (Draft)

The design decisions are yours: what your agent observes, how it is rewarded, and how it is trained. This section describes problems that commonly come up when training a driving agent, and how to recognize them, so you can make those decisions with your eyes open.

### Reward Structure

Your reward function is the only thing reinforcement learning optimizes, and agents are very good at finding unintended ways to collect reward.

`NNVehicle` starts without one. Give rewards (negative ones are penalties) with `Reward(...)`, for each decision in `CollectObservations()` and for wipeouts in `OnWipedOut()`; read their settings from the trainer configuration in `OnEpisodeBegin()` so you can tune them without rebuilding. The points below are what to think about:

- **Balance progress against penalties.** If a wipeout costs much more than the agent can earn by driving for a while, an early agent (which crashes often) may learn that the safest strategy is to barely move: creep along just fast enough to avoid the "stopped" wipeout, never crash, and collect a trickle of reward. This is a stable local optimum that RL can stay stuck in for a very long time. Ask yourself: how many seconds of good driving does it take to earn back one wipeout?
- **Be careful with per-step time penalties.** A small penalty every step seems like a good way to discourage crawling, but when a wipeout ends the episode, crashing quickly can then become the highest-scoring option.

- **Reward the right kind of progress.** Rewarding raw speed can reward driving in circles or sliding sideways. Progress measured along the track (for example, the change in `Racetrack.TotalDistanceTravelled`) rewards actually going down the road, and naturally rewards taking a tighter line through a turn.
- **Ending the episode at a wipeout, or not.** If a wipeout ends the episode, the agent loses whatever it would have earned afterwards; if the car resets and the episode continues (`wipeout_ends_episode: 0`), a wipeout also costs the time spent getting back up to speed, as in a race. The second tends to give a more steady signal for a racing task.
- **Keep reward magnitudes reasonable.** Per-decision rewards are typically small, with episode totals in a range of roughly tens, not thousands. When combining reward signals (extrinsic, GAIL), each has a `strength` in the trainer configuration; their relative sizes matter more than their absolute values.

### Learning from Your Fuzzy Logic Driver (Imitation)

You can record your FL controller driving and use the recordings to train your network much faster than with reinforcement learning alone (behavioral cloning and GAIL in the trainer configuration).

- **Record variety, not just a perfect lap.** Your network will end up in situations your FL driver never visited (after a wipeout, slightly off the racing line, starting from rest). The recording tool periodically resets the car, sometimes offset or turned, and adds small random disturbances to the controls, so the recordings show your FL driver recovering.
- **Only good driving is kept.** Imitation copies whatever is in the recordings, mistakes included. The recording tool therefore drops the seconds leading up to each wipeout (the mistake happens a few seconds before the wipeout registers) and stretches where the car stalls, has wheels off the road, or faces the wrong way. An occasional wipeout under the added disturbances is expected.
- **A slower, safe teacher beats a fast one that crashes.** Read the report printed after recording. If it warns that your controller wipes out or struggles often, record with a speed cap, e.g. `nntools record --teacher-max-speed 60`: above the cap your controller's throttle is limited, and the limited decisions are what get recorded. Imitation only needs to get your network driving; reinforcement learning can add speed afterwards, but it has to unlearn copied crashes.
- **Imitation helps most early on.** Behavioral cloning gets the network driving like your FL controller quickly. If you want the network to eventually do *better* than your FL controller, imitation needs to fade so reinforcement learning can take over.
- **Watch for forgetting.** When imitation fades, a policy that drove well can get worse again. Keep checkpoints, and evaluate several of them rather than assuming the last one is the best.
- **GAIL rewards looking like the demonstrations.** It can make an agent overly cautious (it is rewarded for staying alive and looking similar). The ML-Agents documentation suggests using it at a low strength alongside your own reward.
- **Recordings store observations.** If you change what your agent observes, record new demonstrations.

### Training in Two Phases

Imitation alone can at best drive like your FL controller. To go further, train in two runs:

1. **Imitation** (`Racecar_imitation.yaml`): behavioral cloning and GAIL from your demonstrations, and no task reward, so it learns from your FL driver alone and works before you have designed a reward. (The reward the trainer prints meanwhile is your `NNVehicle`'s, which this phase does not use.) The run ends when behavioral cloning does (`max_steps` equals `behavioral_cloning.steps`): in testing, the policy got clearly worse when training went on past that. Test a few checkpoints (`nntools test --nn --run imitation --steps 200000 300000 400000`).
2. **Reinforcement learning** (`Racecar_rl_after_imitation.yaml`), starting from the imitation policy: `-- --initialize-from <imitation run-id>`. Your reward alone now drives learning. ML-Agents warns that it did not find the `extrinsic` value head and did not expect the `gail` one; that is expected (the imitation run learned from GAIL instead of a task reward), and it starts that part fresh.

`--initialize-from` starts from the run's *last* checkpoint. To start from an earlier one that tested better, also set `init_path` under `behaviors: Racecar:` in the configuration to its file name in `results/<imitation run-id>/Racecar/` (e.g. `init_path: Racecar-299981.pt`). This matters: in testing, reinforcement learning started from a degraded last checkpoint stayed several points behind the same run started from the best one, at every checkpoint.

If your FL controller is not a good teacher even with a speed cap, `Racecar.yaml` (reinforcement learning only) is a valid path; it just takes longer to get started.

Two trainer settings matter a lot for driving:

- `gamma` (discount): how far ahead rewards count, roughly $1/(1-\gamma)$ decisions. At 30 decisions per second, 0.998 is about 17 seconds, enough to see that braking now pays off in speed out of the turn.
- `lambd` (GAE $\lambda$): how much credit for an action comes from what actually happened over the following seconds rather than from the learned estimate of how good the situation was. Higher values (0.99) help the agent connect an early decision (braking before a turn) with its later payoff.

### Choosing and Keeping a Model

Progress is not steady, and a policy can get worse again after improving (for example, getting faster on one track while starting to crash on another). Keep many checkpoints (`keep_checkpoints`), watch or evaluate several of them, and keep the best rather than assuming the last is best. When a run levels off or starts to slip, continuing from a good checkpoint with a smaller learning rate changes the policy more gently.

A trained model can be run with or without its built-in randomness. During training, and by default in Unity, the agent samples each action around its most likely one; with `Deterministic Inference` enabled in `Behavior Parameters`, it always takes the most likely action. Try your model both ways: one that only stays on the road without the randomness is driving close to the edge.

### Rewinding Wipeouts

The `WipeoutRewind` component on the `RacecarNNAgent` prefab is off unless the trainer configuration sets `rewind_fraction` in its `environment_parameters`;

`Racecar_rl_after_imitation.yaml` sets 0.5 (set 0 to turn it off). After that share of wipeouts, the car goes back to a moment a little before the mistake, at the speed it had then, instead of restarting at rest, so the hard spots (a tight turn after a straight, for example) get practiced much more often. Use it with `wipeout_ends_episode: 0`. Rewinding sets the race distance back, so a progress reward computed from changes in `Racetrack.TotalDistanceTravelled` (see *Reward Structure*) automatically takes back the reward for the distance lost. It only acts while training.

### The Copycat Problem

Be careful about letting your network observe its own previous actions (last steering or throttle), or quantities that closely track them (such as the current wheel steering angle, which follows the steering command with a delay).

A driver's controls change smoothly: the steering this step is almost always very close to the steering last step. When learning from demonstrations, the network can discover that "do what you did last step" predicts the demonstrated actions very well, and learn that shortcut instead of learning *why* the controls change (a turn coming up, drifting toward the edge). Training looks great, but when the network drives on its own it keeps repeating its own decisions, reacts late, and runs off the road. This is a known issue in imitation learning (sometimes called the *copycat problem*, an example of *causal confusion*).

Information about the car's actual physical state (speed, sliding, wheel slip) is generally fine; the concern is inputs that mostly echo the previous command.

### Observations

- Keep observations roughly in $[-1, 1]$ by dividing each by a fixed, typical largest value (`NNVehicle`'s `Observation Scales`). Avoid relying on the trainer's `normalize` option for this: it rescales by running statistics of what the agent has seen so far, which shift as its driving changes (e.g. between imitation and reinforcement learning), so the same situation can look different to the network later on. In testing, this made a policy that drove well stop moving at standing starts. The report after `nntools record` lists each crisp input's range and a typical largest value, and flags observations the network sees far outside $[-1, 1]$ or barely varying.
- Express positions and directions relative to the car (in the car's frame) rather than in world coordinates.
- Prefer inputs that change smoothly. For example, sampling the road at fixed distances ahead changes smoothly as the car moves, whereas "the next five Bézier segments" jumps every time the car passes a segment boundary.
- A natural starting point is the same crisp inputs your FL controller uses, but you are not limited to them. Your network can do no better than its inputs allow, so think about what else would help (for example, how the road bends further ahead, or how much the tires are sliding). Add those in `NNVehicle`'s `CollectObservations()` with `sensor.AddObservation(...)`, already scaled to roughly $[-1, 1]$ (`Observation Scales` covers only the crisp inputs), and count them in the *Space Size*.
- Your FL controller does not need to use those extra observations. While you record, your `FuzzyVehicle` drives and the network's agent rides along, computing all of its observations, so the recordings pair everything the network observes with what your FL driver did. What

imitation does need is for the observations to include whatever your FL driver reacts to. If something is missing, the network sees the same inputs in situations where your FL driver did different things, and it can only learn an average of them.

### Reading Your Training Graphs

TensorBoard (see *Train* above) shows how training is going. The mean reward the trainer prints every 20k steps is much noisier than it looks: it averages only the episodes that ended since the last line, often just one or two, and with `--scene auto` each copy of the game is a different track with its own typical episode reward (the drag race's is several times a curvy track's). Look at the smoothed trend in TensorBoard, and judge a model by testing it. Some patterns to recognize:

| What you see | Possible cause |
|---|---|
| Episodes run to the time limit but reward stays near zero | The agent learned to crawl (see Reward Structure) |
| Behavioral cloning loss keeps dropping, but episodes stay short (frequent wipeouts) | Copycat problem, or observations that don't capture what the FL driver reacts to |
| Reward rises, then falls after imitation fades | Forgetting; use an earlier checkpoint, keep some imitation, or lower the learning rate |
| Reward stays at the wipeout penalty for a long time | The agent never gets far enough to discover reward; consider imitation or rebalancing rewards |

## Summary

This optional task allows you to experiment with neural network control and compare it against fuzzy logic control. Successful training may require tuning the observation set, rewards, and training hyperparameters. While not graded, this experience can provide valuable insight into reinforcement learning methods and their tradeoffs in dynamic control tasks such as the Racetrack environment.